

CSI243: Functional programming

When is test 1?

week 7 lect 1

Defining functions

- **New definitions using existing functions**
 - **Examples (specify and implement):-**
 - **Is it a digit**

c	isDigit(c)
'd'	$'0' \leq c \leq '9'$
'3'	$'0' \leq c \leq '9'$
'x'	$'0' \leq c \leq '9'$
i	$'0' \leq c \leq '9'$
(i + 1)	$'0' \leq c \leq '9'$

CSI243: Functional programming

Defining functions

- **New definitions using existing functions**
- **Examples (specify and implement):-**

- **Is it a digit**

`isDigit : Char → Boolean`

`isDigit(Char c):Boolean`

Purpose:

Prec:

Postc: (`'0' ≤ c ≤ '9'`)

CSI243: Functional programming

Defining functions

- **New definitions using existing functions**
 - **Examples (specify and implement):-**

- **Is it a digit**

```
isDigit :: Char -> Bool
```

```
isDigit c = ('0' <= c) && (c <= '9')
```

- **Is it an odd integer**

```
isOdd :: Integral a => a -> Bool
```

```
isOdd c = (mod c 2) /= 0
```

- **Split a list at the nth element**

```
splitat :: Int -> [a] -> ([a], [a])
```

```
splitat n xs = (take n xs, drop n xs)
```

CSI243

Defining functions

- **New definitions using existing functions**

- **Examples:-**

- **Invert an integer**

```
invert :: Fractional a => a -> a
invert c = recip c
```

- **Conditional expressions**
- **Several ways to implement selection. One of the options is the conditional expression**

```
if (condexp) then st else stm
```

Defining functions

- **Conditional expressions**
 - **Example (determine the sign of a integer)**

n	getSign(n)
-2	$\begin{cases} 1 & \text{iff } (n > 0) \\ 0 & \text{iff } (n = 0) \\ -1 & \text{otherwise} \end{cases}$
1	$\begin{cases} 1 & \text{iff } (n > 0) \\ 0 & \text{iff } (n = 0) \\ -1 & \text{otherwise} \end{cases}$
i	$\begin{cases} 1 & \text{iff } (n > 0) \\ 0 & \text{iff } (n = 0) \\ -1 & \text{otherwise} \end{cases}$
(i + 1)	$\begin{cases} 1 & \text{iff } (n > 0) \\ 0 & \text{iff } (n = 0) \\ -1 & \text{otherwise} \end{cases}$

Defining functions

- **Conditional expressions**
 - **Example (determine the sign of a integer)**

getSign : $\mathbb{Z} \rightarrow \mathbb{Z}$

getSign(int n) : int

Purpose: ...

Prec:

Postc:

$$= \begin{cases} 1 & \text{iff } (n > 0) \\ 0 & \text{iff } (n = 0) \\ -1 & \text{otherwise} \end{cases}$$

Defining functions

- **Conditional expressions**
 - **Example (determine the sign of a integer)**

```
getSign :: Integral a => a -> a
getSign n =
    if (n < 0) then -1 else
    if (n > 0) then 1  else 0
```

Defining functions

- **Conditional expressions**
 - **Example (determine the sign of a integer)**

```
getSign :: Integral a => a -> a
getSign n =
    if (n < 0) then -1 else
    if (n > 0) then 1 else 0
```

- **Guarded equations**
 - **Alternative to the use of conditional expressions**
 - **A sequence of logical expressions called guards is use to choose between a sequence of results of the same type**

Defining functions **CSI243**

- **Guarded equations**
 - **Follow Haskell indentation rule**
 - **Example**
- **sign** =
$$\begin{cases} +ve \text{ iff } n > 0 \\ -ve \text{ iff } n < 0 \\ 0 \text{ otherwise} \end{cases}$$
- **If the 1st guard is true, the 1st result is chosen, ...**

```
getSign :: Integral a => a -> a
```

```
getSign n
```

```
    | (n < 0) = -1
```

```
    | (n > 0) = 1
```

```
    | otherwise = 0
```

- **Otherwise guard optional, but good to have**

see

- <http://book.realworldhaskell.org>
- <http://www.json.org/>